Equivalent definitions of a tree

For a finite nonempty simple undirected graph on $n \geq 1$ vertices, these are equivalent: connected and acyclic; connected with $n-1$ edges; acyclic with $n-1$ edges; exactly one simple path between every two vertices. A forest only requires acyclicity and may have several components. A single vertex has zero edges, and its unique self-path has length zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proofs of the equivalences

An endpoint of a longest tree path is a leaf: an outside neighbor would extend the path, while another neighbor on the path would create a cycle. For $n \geq 2$, deleting a leaf and its edge preserves connected acyclicity. Induction from one vertex proves $e = n - 1$. Applying this to every forest component gives $e = n - c$.

Any connected graph can have cycle edges deleted until a spanning tree remains, so it has at least $n - 1$ edges. If it initially has exactly $n - 1$, no deletion was possible and it was acyclic. An acyclic graph with $n - 1$ edges has $c = n - e = 1$, hence is connected.

A tree has a path by connectivity. Two distinct simple paths would yield a cycle between their first divergence and next common vertex, contradiction. Conversely, unique paths imply connectivity and exclude cycles, whose two directions would give distinct paths. This proves all equivalences without assuming the edge-count result in advance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Leaf counts and boundary conventions

For $n \geq 2$, let n_i count degree-i vertices. Subtract $\sum n_i = n$ from $\sum i n_i = 2(n-1)$ to obtain

$$n_1 = 2 + \sum_{i \geq 3} (i-2) n_i.$$

Thus at least two leaves exist. Degree-two subdivision vertices do not change the leaf count. Two degree-three vertices and one degree-four vertex force $2+2+2=6$ leaves, regardless of degree-two vertices.

The singleton has no degree-one leaf, though it is a rooted leaf under the no-children definition. The formula assumes $n \geq 2$ and undirected degree-one leaves. The empty graph does not satisfy $e = n-1$; trees here are nonempty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bridges and fundamental cycles

In a connected graph an edge is a bridge iff it belongs to no cycle. Consequently every tree edge is a bridge, and adding an edge to a tree creates exactly one fundamental cycle. A triangle with a dangling edge has both nonbridges and a bridge; being a tree also requires connectivity and every edge to be a bridge. Removing a bridge increases components; removing a nonbridge does not disconnect its component.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rooted trees, depth, and height

A root determines parent, children, and ancestors. Vertex depth is its root distance in edges; height is maximum leaf depth; level d contains depth-d nodes. An m-ary tree has at most m children per internal node; a full m-ary tree has exactly m. Binary trees also distinguish left and right. Fix whether singleton height is zero or one before using formulas; here root depth is zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ordered trees and addresses

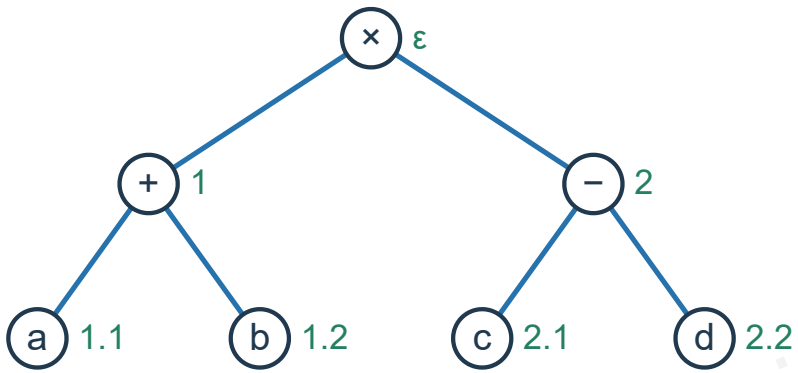
An ordered rooted tree additionally orders each node's children. Append child position i to its parent's address; the root is ε. Address 2.1 has parent 2 and ancestors ε,2; address length is depth. Address numbers are child positions, not key comparisons.

For $(a+b)\times(c-d)$, the root $\times$ has children +,− and ordered leaves a, b, c, d. Preorder visits root before children; postorder after children; binary inorder visits left, root, right. General ordered trees have no unique inorder convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ordered trees and addresses (continued)

One ordered binary tree: addresses,subtrees,and evaluation



Root ε has depth 0; leaf addresses have length and depth 2.

Preorder：× + a b − c d

Parenthesized inorder: (a+b)×(c−d)

Postorder：a b + c d − ×

Substitute 2,3,9,4: result 25.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Addresses, traversals, and evaluation

Node	×	+	a	b	−	c	d
Address	ε	1	1.1	1.2	2	2.1	2.2
Depth	0	1	2	2	1	2	2

Preorder `× + a b − c d` is prefix; postorder `a b + c d − ×` is postfix. Parenthesized inorder gives `(a+b)×(c−d)`. Removing parentheses yields `a+b×c−d`, generally a different expression; traversal alone does not preserve precedence.

With a=2,b=3,c=9,d=4, postorder computes 2+3=5,9−4=5,then 25. Each node has constant work and each edge is traversed at most twice, giving Θ(n) traversal time and O(h+1) recursion stack, worst-case O(n) in a skew tree.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Binary-tree counts

A full binary tree has exactly two children at every internal node and satisfies $L = I + 1$. With root depth zero and height at most h,

$$I \le 2^h - 1, \qquad L \le 2^h.$$

Both equalities hold only for a perfect tree with every level full and all leaves at depth h. A full tree need not be perfect: give the root one leaf child and one internal child with two leaves. Its height is 2, I=2, and L=3, not I=3 and L=4. Catalan numbers count shapes; applications such as Merkle trees add labels and ordering requirements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full, perfect, complete, and m-ary trees

Full means every internal node has exactly m children; perfect additionally puts all leaves at the same depth. Complete binary means all but the final level full and the last filled left to right. Some references call full binary trees complete; use the structural definition.

For full m-ary trees, $m \geq 2$, edge counting gives

$$mI = I + L - 1, \qquad L = (m-1)I + 1, \qquad n = mI + 1.$$

With at most m children and height h, level d has at most m^d nodes, total at most $(m^{h+1}-1)/(m-1)$, and leaves at most m^h. The leaf bound follows by height induction over at most m subtrees, each with at most m^{h-1} leaves. Equality requires full levels to one height, not mere fullness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary shapes and Catalan convolution

Let C_n count full ordered binary shapes with n internal nodes; $C_0=1$ is a single leaf. For $n \geq 1$, the root uniquely leaves i internal nodes on the left and $n-1-i$ on the right, so

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, \qquad C_0, C_1, C_2, C_3 = 1, 1, 2, 5.$$

Chapter 10 gives $C_n = \binom{2n}{n}/(n+1)$. Encode internal nodes as (and leaves as) in preorder, deleting the final leaf symbol. This gives n balanced pairs: open child slots cannot be exhausted before the final node, ensuring nonnegative prefix balance. Conversely fill the next empty slot from each symbol and restore the final leaf, uniquely recovering the shape. This is a bijection, not just matching initial values. Swapping left/right usually changes the shape; removing order would make this product classification repeat objects.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Comparison-sorting decision trees

A comparison sort on distinct keys has a binary decision tree, with comparisons at internal nodes and output orders at leaves. It must distinguish n! orders. Height H permits at most 2^H leaves, so $H \geq \log_2(n!) = \Omega(n \log n)$. This is a comparison-model lower bound; counting and radix sorts use additional key assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worst-case and average comparison lower bounds

The estimate $n! \geq (n/2)^{n/2}$ already proves Ω(n log n) without Stirling. This lower bound concerns some input, not every input.

For N=n! equiprobable orders with leaf depths d_i, prefix-free leaf addresses give $\sum_i 2^{-d_i} \leq 1$. Arithmetic–geometric mean gives

$$2^{-\frac{1}{N}\sum_i d_i} \leq \frac{1}{N}\sum_i 2^{-d_i} \leq 1/N.$$

Thus average comparisons are at least $\log_2 N$ too. If comparisons offer equality but keys are distinct, that branch is unreachable and only two branches count. Nonuniform orders require their actual distribution's entropy bound, not the uniform result.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Prefix codes

A binary encoding C must first be **injective**: different source symbols receive different bit strings. Under this condition it is prefix-free if no codeword is a proper prefix of another. Equivalently, for distinct symbols a,b, C(a) is not a prefix of C(b), including equality.

Injectivity cannot be omitted. If a and b both receive 0, no two distinct codeword strings form a proper-prefix pair, yet receiving 0 does not identify the symbol. For a valid prefix code, store one unique symbol at each binary-trie leaf; read bits from the root until a leaf, output its symbol, and restart at the root. For frequencies f_i, mean length is $L = \sum_i f_i |w_i| / \sum_i f_i$. Prefix-freeness ensures immediate unique decoding but is not necessary for every uniquely decodable code.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Kraft's inequality and its converse

For finite prefix-code lengths ℓ_i and maximum H, a word at depth ℓ_i occupies $2^{H-\ell_i}$ descendant leaves at depth H. These sets are disjoint, so $\sum_i 2^{H-\ell_i} \leq 2^H$ and $\sum_i 2^{-\ell_i} \leq 1$.

Conversely sort nonnegative requested lengths. At depth ℓ, previous codewords cover $\sum_{j<i} 2^{\ell-\ell_j}$ nodes. The Kraft partial sum including the next request guarantees an available node. Select it outside earlier subtrees and forbid its descendants thereafter. This constructs the prefix code.

Lengths 1,2,3,3 sum to Kraft one and permit `0,10,110,111`. Lengths 1,1,2 sum to 5/4 and are impossible. Kraft concerns feasible lengths, not arbitrary proposed strings: `0,00` still conflict.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman: merge the two smallest weights

For at least two symbols with positive frequencies f_i, a binary prefix tree has word lengths equal to leaf depths ℓ_i. Minimize weighted external path length $W = \sum_i f_i \ell_i$. With fixed frequencies, this also minimizes mean word length.

Start with one leaf per symbol. Repeatedly extract the two lightest trees from a min-priority queue, join them below a new parent, and reinsert their total weight. Stop at one tree and label each pair of child edges 0 and 1. An internal weight is a subtree frequency total, not a new source symbol.

Minimum height need not mean minimum W. Longer paths for rare symbols may buy shorter paths for frequent ones. Correctness needs two arguments: why may the two smallest weights be siblings, and why is solving the contracted problem sufficient?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exchange lemma: moving a smaller weight deeper

A unary internal node cannot occur in an optimum: contracting it decreases every descendant depth by one and strictly reduces positive-weight cost. Use a full binary tree. A deepest leaf at depth H has a leaf sibling, since an internal sibling would have a child deeper than H.

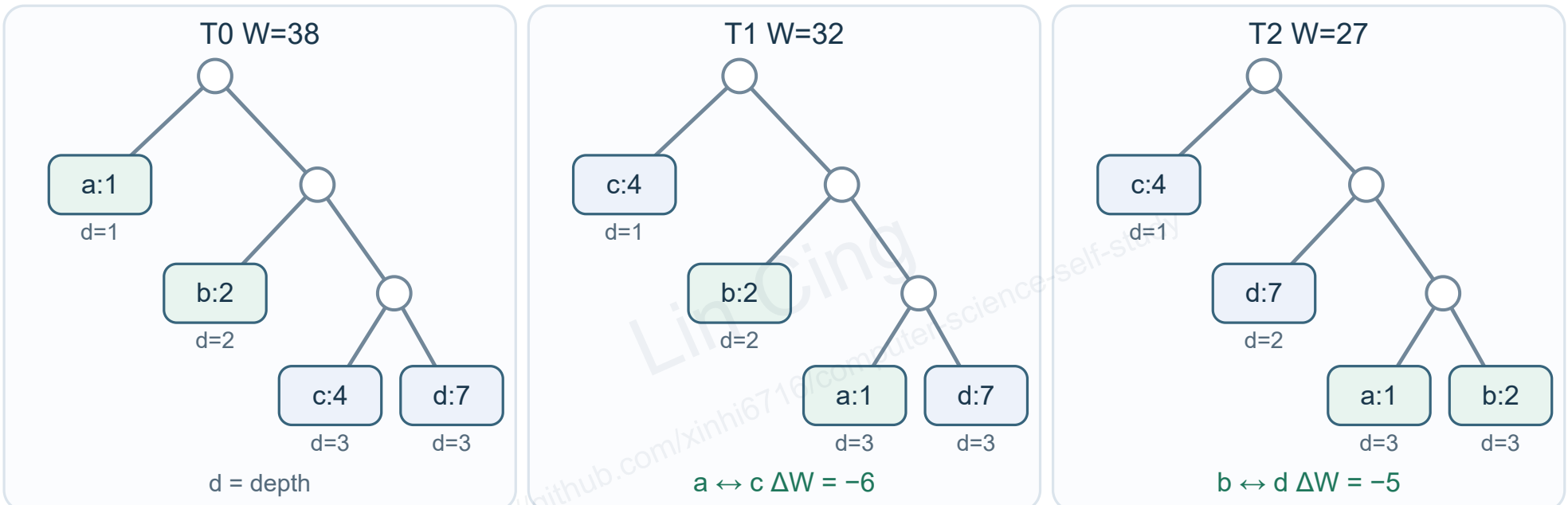
Let $f_x \le f_u$, with x initially at depth h and u at $H \ge h$. Swap only their leaf labels. Every other term stays fixed, so

$$\Delta W = (f_x H + f_u h) - (f_x h + f_u H) = (f_u - f_x)(h - H) \le 0.$$

Put the minimum-weight symbols x,y in those deepest sibling positions. If one already occupies a target position, keep it and fill the other. Otherwise place x first, then y without moving x. Every remaining symbol weighs at least f_y, so both swaps satisfy the inequality. Starting from an optimum forces equality: a smaller cost would contradict optimality. Some optimum accommodates the chosen merge; optimal trees need not all coincide.

A visible exchange: 38 → 32 → 27

Swap leaf labels; preserve the tree shape



This deliberately starts from a nonoptimal cost-38 tree. Only leaf labels move; edges and leaf positions stay fixed. The minimum weights a,b end as siblings at depth three, yet cost 27 exceeds Huffman's 24. The exchange lemma guarantees nonincrease; two exchanges do not optimize an arbitrary tree.

Which two contributions change in each swap?

Fix frequencies a:1, b:2, c:4, d:7. Each entry gives depth and its frequency-times-depth cost.

Symbol	Original T_0	Swap a,c: T_1	Swap b,d: T_2
a:1	Depth 1; cost 1	Depth 3; cost 3	Depth 3; cost 3
b:2	Depth 2; cost 4	Depth 2; cost 4	Depth 3; cost 6
c:4	Depth 3; cost 12	Depth 1; cost 4	Depth 1; cost 4
d:7	Depth 3; cost 21	Depth 3; cost 21	Depth 2; cost 14
W	38	32	27

First a pays two more and c pays eight less: $\Delta W = (4-1)(1-3) = -6$. Next b pays two more and d pays seven less: $\Delta W = (7-2)(2-3) = -5$. Equal weights or equal depths give zero difference. Distinct symbols must still receive distinct words.

Contracting siblings removes one final level

Replace sibling leaves x,y and their parent by leaf z of weight $f_z = f_x + f_y$. If z has depth k in the contracted tree U, expansion places x,y at k+1. All other leaf depths stay fixed.

$$f_x(k+1) + f_y(k+1) = (f_x + f_y)k + f_x + f_y.$$

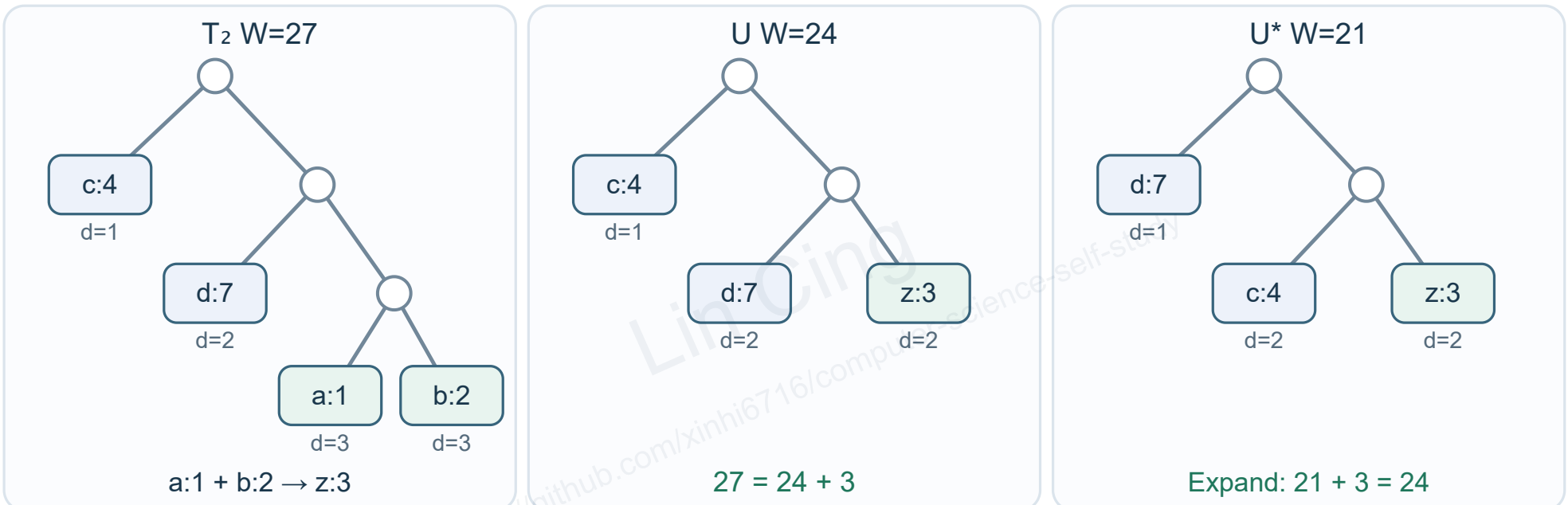
Every tree T with sibling x,y satisfies $W(T) = W(U) + f_x + f_y$. Conversely any legal reduced-problem tree U expands at z into a legal original prefix tree, adding exactly this constant.

If T is optimal, U must be optimal too. Otherwise replace U by a cheaper U* and expand the same pair: the original cost becomes smaller, contradicting optimality of T. This establishes optimal substructure rather than assuming local choices solve the whole problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Contraction example: 27 − 3 = 24, then improve

Contract, then solve the reduced problem



Contracting depth-three a,b replaces cost 9 by 6, so U costs 27−3=24. This is reduced cost, not original cost. For weights 3,4,7, depths 2,2,1 give W(U*)=21. Expanding z recovers cost 21+3=24 and depths 3,3,2,1 for a,b,c,d. Reoptimizing U supplies the improvement missing after the swaps.

Finish the optimality proof by induction

Induct on symbol count n with positive weights. A singleton may use the empty word at cost zero; two symbols use one bit each at cost their weight sum. Assume Huffman is optimal for all smaller alphabets.

Merge chosen minimum-weight x,y into z. The exchange lemma gives some original optimum T* with sibling x,y. Its contraction U* is optimal for the reduced alphabet by the replacement argument. Consequently,

$$\mathrm{OPT}(C) = \mathrm{OPT}(C') + f_x + f_y, \qquad C' = (C \setminus \{x, y\}) \cup \{z\}.$$

Huffman continues on C', attaining $\mathrm{OPT}(C')$ by induction. Expansion at z attains the original optimum. Each merge reduces the symbol count by one. Tied minimum weights and swapped 0/1 edges are allowed; the theorem neither promises unique codewords nor imposes a fixed alphabetic leaf order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman example with weights 1,2,4,7

Merge 1+2=3,then 3+4=7,then 7+7=14. The weighted external path length is 3+7+14=24. One code assigns frequency 1:`110`,2:`111`,4:`10`,7:`0`, giving average 24/14. No prefix code has lower weighted cost. Edge labels may be swapped if the decoder uses the same trie.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman cost model and boundary cases

Assume positive frequencies and objective $W=\Sigma f_i \ell_i$. An optimal tree has no unary internal node: contraction decreases every positive-weight descendant depth and strictly improves W. With at least two symbols, full binary trees therefore suffice.

A min-priority queue performs $n-1$ merges in $O(n \log n)$ time and $O(n)$ space. Sorted frequencies permit two nondecreasing queues, one for original leaves and one for new nodes, selecting each minimum from their heads in $O(n)$ merge time. Include sorting if input is unsorted.

One symbol may use ε with cost zero if its count is separately known; a format requiring nonempty words instead uses 0 at cost f. Zero-frequency symbols do not affect the objective; remove them or specify required encoding, since the strict-improvement proof uses positivity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why merge sums equal external path length

Leaf i contributes f_i to each ancestor's subtree weight, once per depth step. Exchanging finite sums gives

$$\sum_i f_i \ell_i = \sum_{\text{internal } v} \text{weight}(v).$$

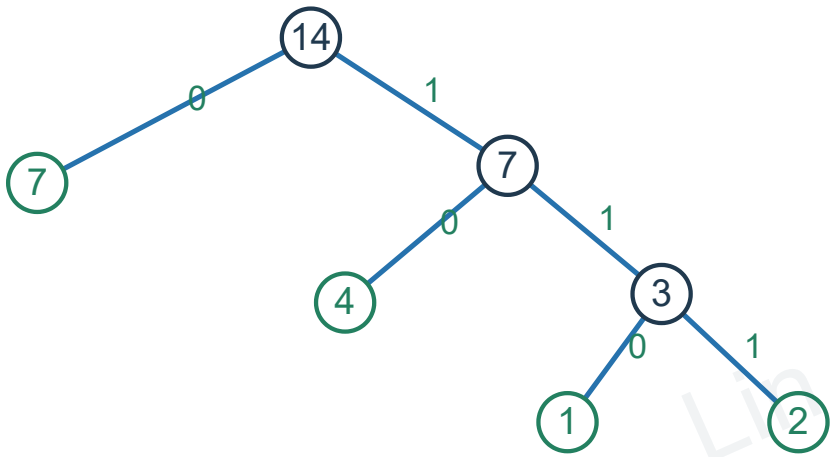
Round	Extract	Insert	Remaining queue
1	1,2	3	3,4,7
2	3,4	7	7,7
3	7,7	14	14

Depths 3,3,2,1 give 1 • 3+2 • 3+4 • 2+7 • 1=24=3+7+14. Mean 24/14≈1.7143 bits is below fixed two-bit coding, excluding codebook storage.

Lin Cing (https://github.com/xinhi6716/computer-science-self-study)

The actual prefix-code tree

Huffman weights,codewords,and weighted cost



Weight	code	Length	Cost
7	0	1	7
4	10	2	8
1	110	3	3
2	111	3	6

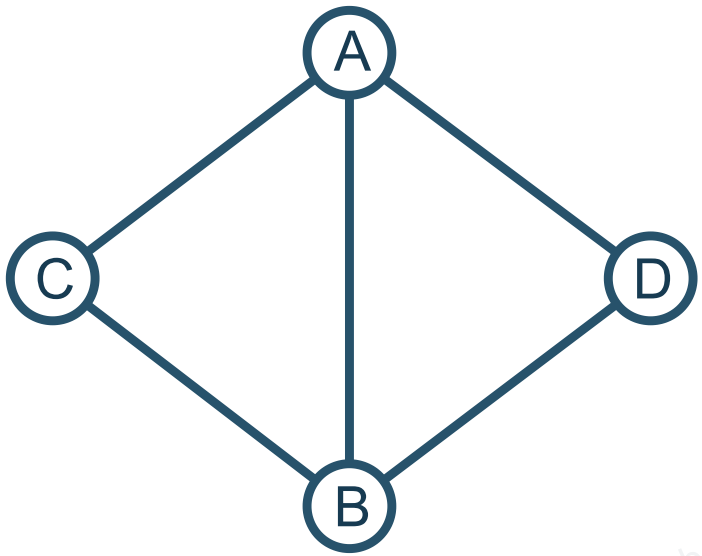
Total cost 24; mean length 24/14 bits.

Internal nodes hold subtree frequency totals; edges hold bits; only leaves represent symbols. A root-to-leaf path spells a codeword. An internal weight seven and a leaf frequency seven may coincide numerically, but treating the internal node as a source symbol would make its code a prefix of descendant codes.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Counting spanning trees with the Laplacian

Consider K_4 after deleting CD. How many spanning trees include all four vertices?



Five edges

AB, AC, AD, BC, BD

Degrees A, B, C, D: (3, 3, 2, 2)

CD is absent

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The matrix–tree theorem and incidence matrix

For a finite simple undirected graph with $n \geq 1$ vertices, orient each edge arbitrarily. In its incidence matrix B, an edge column has -1 at its tail, $+1$ at its head, and zero elsewhere. Its Laplacian is

$$L = BB^{\mathsf{T}} = D - A.$$

Matrix–tree theorem: deleting row and column r of L gives $L^{(r)}$, and $\det L^{(r)}$ equals the number $\tau(G)$ of spanning trees. A disconnected graph gives zero; for one vertex the empty determinant is 1.

The orientations only set signs in B: reversing an edge negates one column and leaves BB^{T} unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why does the cofactor count trees?

Delete row r from B to obtain B_r, so $L^{(r)} = B_r B_r^{\mathsf{T}}$. Cauchy–Binet sums over sets S of $n-1$ edges:

$$\det L^{(r)} = \sum_{|S|=n-1} \det(B_r[:, S])^2.$$

If S contains a cycle, signed incidence columns around that cycle sum to zero, so its determinant vanishes. If S is an acyclic disconnected forest, its $n-c$ edges cannot equal $n-1$. Thus only spanning trees can contribute.

For a tree, repeatedly choose a leaf other than r. Its row has just one nonzero entry, ± 1; expand on that row and remove its edge. The remaining graph is a tree, ending with empty determinant 1. Every tree therefore contributes $(\pm 1)^2 = 1$, so the sum counts all spanning trees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked example: K_4 without CD

In vertex order A, B, C, D, the five edges are AB, AC, AD, BC, BD. Their degrees are $3, 3, 2, 2$, giving

$$L = \begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 2 & 0 \\ -1 & -1 & 0 & 2 \end{pmatrix}, \qquad L^{(D)} = \begin{pmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 2 \end{pmatrix}.$$

Expanding along the first row gives three signed contributions:

$$3(6-1) \; - \; (-1)(-2-1) \; + (-1)(1+3) = 15 - 3 - 4 = \boxed{8}.$$

Thus this five-edge graph has 8 spanning trees; it is not the complete K_4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search-tree sorting and its limits

With strict BST invariant left keys<key<right keys, inorder outputs sorted keys; duplicate handling needs its own policy. Sorted insertion without balancing can produce height n−1 and Θ(n) worst-case search. Balanced trees maintain Θ(log n) height through valid rotations and invariants, not visual appearance. Decision-tree lower bounds do not imply that every tree-based sorting algorithm attains optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS and BFS spanning forests

In each component choose a root and retain the first-discovery edge for every new vertex. Each nonroot has one parent; discovery time strictly increases along parent edges, preventing cycles, and every discovered vertex reaches the root. The result is a spanning forest, or a tree when connected.

DFS finishes one neighbor subtree recursively before the next. BFS uses a FIFO queue and nondecreasing distances. Setting d[w]=d[v]+1 on discovery is shortest: a shorter path's penultimate vertex would leave the queue earlier and discover w earlier, contradiction. DFS depth generally is not shortest distance.

For edges AB, BC, CA, CD, DE, EF, FD, DG with alphabetic neighbors:

Method	Discovery order	Tree edges
DFS	A, B, C, D, E, F, G	AB, BC, CD, DE, EF, DG
BFS	A, B, C, D, E, F, G	AB, AC, CD, DE, DF, DG

Equal discovery orders do not imply equal trees. C has BFS depth one and DFS depth two. Adjacency lists cost $O(n+m)$; matrices scan n entries per vertex and cost $\Theta(n^2)$ in the worst case.

Undirected DFS has no sibling cross edges

For a nontree edge xy, assume x is discovered first. If y is undiscovered before x returns, scanning xy puts y in x's subtree. If y was already discovered, it must be an active ancestor: a completed subtree would previously have scanned yx and discovered x, contradicting the later discovery. Thus every nontree edge joins ancestor and descendant.

This underlies low-link proofs. Separate child subtrees can connect around their common parent only through ancestor back edges, not arbitrary cross edges. Directed DFS may have cross edges; these undirected formulas do not directly solve directed strongly connected components.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Merge invariant and complete example

For sorted L, R, keep cursors i, j and initially empty output O. Repeatedly output the smaller remaining head and advance that cursor. Choose the left head on equality for stability. When a side empties, append the other remainder.

Invariant: O is sorted and equals the multiset of removed items, and all remaining items are at least its last item. The smaller head is the minimum of everything remaining, so each step preserves the invariant. Remaining length decreases by one, proving termination and a complete sorted merge.

Merging [2,3,6,8] and [1,4,5,7] outputs 1,2,3,4,5,6,7 by seven comparisons and appends eight. There is no repeated scan of the remainder. Positive lengths p, q require at most p+q−1 comparisons; an initially empty side requires zero.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Merge sort and comparison counts

Split into halves differing by at most one, sort them recursively, then merge. Length zero/one returns immediately. Induction on length gives sorted halves and the merge lemma gives correct complete output.

Stage	Data state	Comparisons
Input	[8,3,6,2,7,1,5,4]	0
Merge singletons	[3,8],[2,6],[1,7],[4,5]	4
Merge pairs	[2,3,6,8],[1,4,5,7]	3+3=6
Final merge	[1,2,3,4,5,6,7,8]	7

Total seventeen. For $n=2^h$, depth-d merges require at most $n-2^d$ comparisons, with root d=0. Thus $M(n) \le \sum_{d=0}^{h-1}(n-2^d) = n\log_2 n - n + 1$, attained here at n=8. General n has depth $\lceil \log_2 n \rceil$ and O(n) work per level, yielding O(n log n); the comparison lower bound makes worst-case Θ(n log n). A reused temporary array uses O(n) extra space; retaining historical copies at every level is a different model.

Articulation points and low values

A cut vertex increases component count on deletion. For DFS discovery times,

$$low[v] = \min\Big(disc[v], \min_{\text{back edge } v \to u} disc[u], \min_{\text{tree child } w} low[w]\Big).$$

The root is a cut vertex iff it has at least two DFS children. A nonroot v is a cut vertex iff some child w has low[w]≥disc[v]. Exclude the parent edge from back edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two different low-link inequalities

For nonroot v, low[w]≥disc[v] means child subtree w has no edge to a proper ancestor of v. With no sibling cross edges, deleting v separates it from the parent side. Conversely, if every child has low[w] <disc[v], each reaches an ancestor and the parent side without v; all remain connected, so v is not a cut vertex.

The root has no parent side. Two or more children separate on its deletion because no cross edges connect them. One child contains all remaining vertices connected by its DFS tree; deleting the root does not increase components. An isolated root's deletion decreases components and is not a cut.

A tree edge vw is a bridge iff low[w]>disc[v]. Equality allows a back edge to v, bypassing vw but not v. Thus the edge need not be a bridge while v may be a cut. The strict and nonstrict inequalities cannot be exchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

General-graph low values and block boundaries

Use IDs 1:AB,2:BC,3:CA,4:CD,5:DE,6:EF,7:FD,8:DG and alphabetic DFS from A.

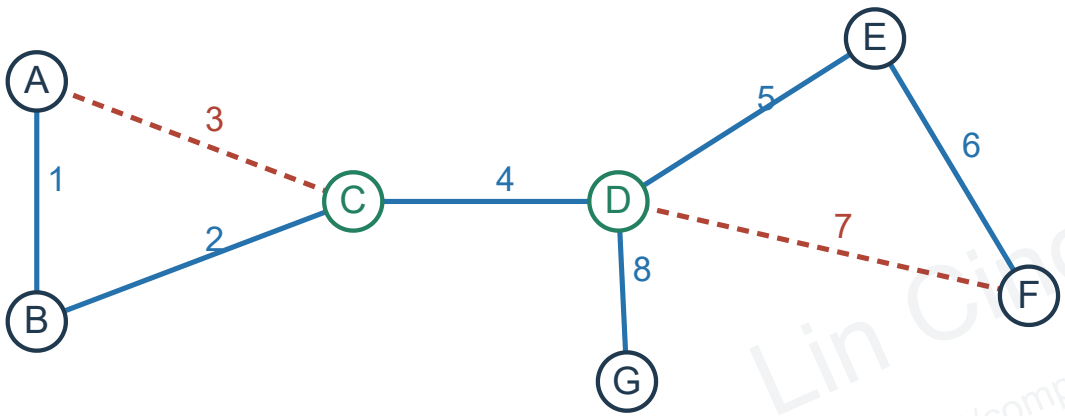
Vertex	A	B	C	D	E	F	G
disc	1	2	3	4	5	6	7
final low	1	1	1	4	4	4	7
parent	None	A	B	C	D	E	D

CA sets low[C]=1 and propagates to B; FD sets low[F]=4 and propagates to E. G has no back edge and low[G]=7. Cuts are C, D; bridges CD, DG. Root A has one child, so even a child's low≥disc[A] does not make it a cut.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

General-graph low values and block boundaries (continued)

Eight-edge DFS: two triangles,two bridges,and low values



Vertex	A	B	C	D	E	F	G
disc	1	2	3	4	5	6	7
low	1	1	1	4	4	4	7

Cuts: C,D; A has one child.

Bridges: 4=CD,8=DG.

Dashed back edges: 3=CA,7=FD.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Actual edge stacks and all four pops

Stacks list IDs bottom to top. Push a back edge only from descendant to ancestor, once.

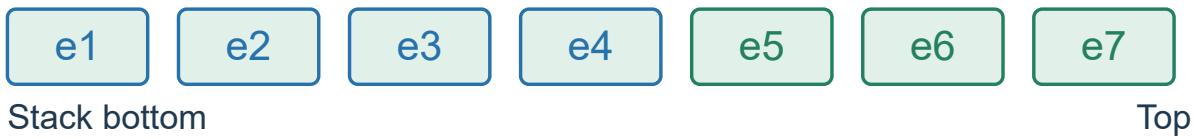
Event	Stack before test or after push	Pop order	Remainder
Enter E subtree and discover FD	1,2,3,4,5,6,7	None	Unchanged
E returns to D: 4≥4	1,2,3,4,5,6,7	7,6,5	1,2,3,4
Push DG; G returns: 7≥4	1,2,3,4,8	8	1,2,3,4
D returns to C: 4≥3	1,2,3,4	4	1,2,3
C returns to B: 1<2	1,2,3	None	1,2,3
B returns to A: 1≥1	1,2,3	3,2,1	Empty

Blocks are {DE, EF, FD},{DG},{CD},{AB, BC, CA}. Every pop includes its triggering tree edge. Even a root not marked as a cut must emit a block when the low condition holds.

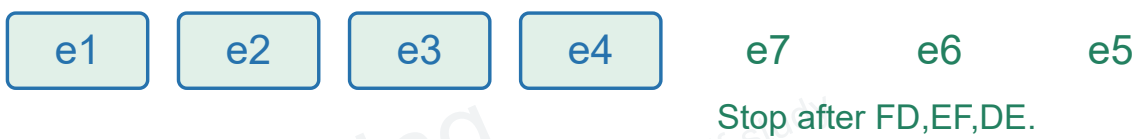
Before, during, and after one pop

Return E→D: pop through DE,retain earlier pending edges

Before: low[E]=4,disc[D]=4



Action: pop e7,e6,e5



After: emit complete DEF block



The trigger is e_5=DE, so stop only after including e_5. Popping only top e_7 would split the triangle; also popping e_4 would wrongly include the bridge to earlier vertices. This operation emits a completed block from pending edges without modifying the graph.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Vertex-biconnected blocks

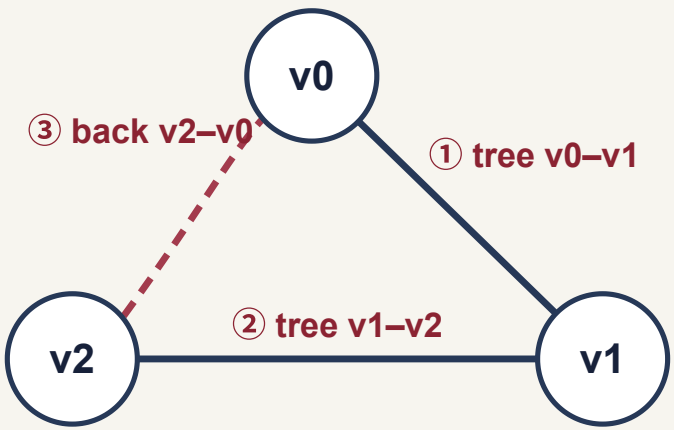
A block is a maximal connected subgraph with no articulation point, with maximality under subgraph inclusion. Connectivity excludes disconnected cut-free sets. This chapter treats a bridge plus endpoints K_2 as a block and an isolated K_1 as a singleton block.

The DFS edge stack receives tree edges and back edges to ancestors. On returning along tree edge vw with $\text{low}[w] \geq \text{disc}[v]$, pop a contiguous suffix through and including vw. The whole group is one block; one popped edge alone is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex-biconnected blocks (continued)

Triangle DFS: discovery and low values



DFS states in visit order

v0 disc=1 low=1

v1 disc=2 low=1

v2 disc=3 low=1

All low values reach root v0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

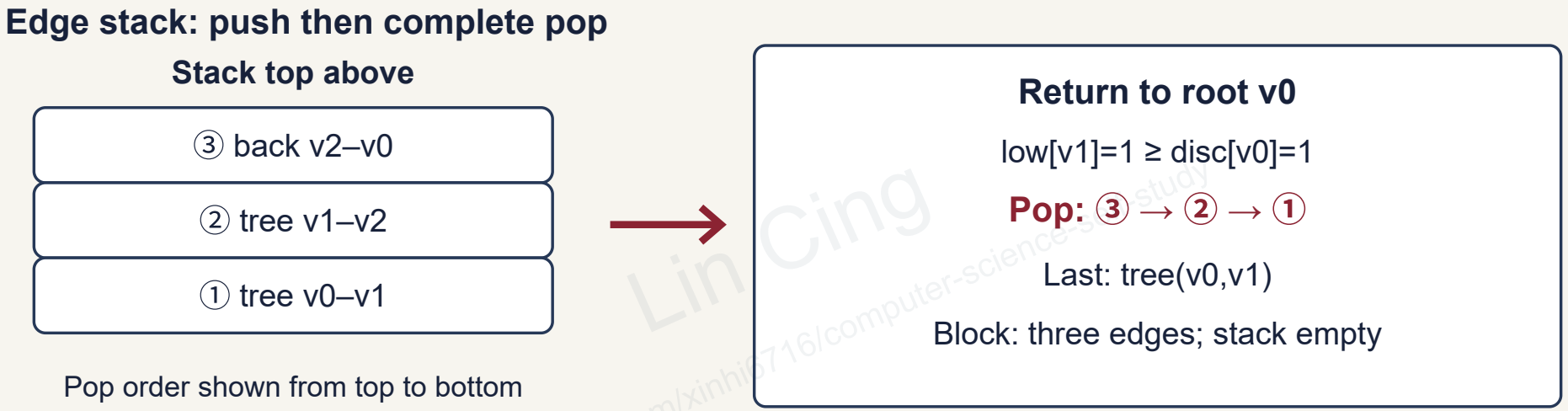
Full stack trace for one triangle

DFS visits v_0, v_1, v_2 then the back edge v_2v_0.

Step	Action	disc	low	Stack	Pop
1	Visit v_0	v_0:1	v_0:1	Empty	None
2	Tree v_0v_1	v_1:2	v_1:2	(v_0,v_1)	None
3	Tree v_1v_2	v_2:3	v_2:3	$(v_0,v_1),(v_1,v_2)$	None
4	Back v_2v_0	Unchanged	v_2:min(3,1)=1	Append (v_2,v_0)	None
5	Return to v_1	Unchanged	v_1:min(2,1)=1	Unchanged	None: $1<2$
6	Return to v_0	Unchanged	v_0:1	Unchanged	All three: $1\geq 1$

Stack suffixes and block boundaries

Step six pops $(v_2,v_0),(v_1,v_2),(v_0,v_1)$, ending at the triggering tree edge. These form the full block and empty the stack.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stack suffixes and block boundaries (continued)

An unfinished subtree owns a continuous stack suffix. If low[w]<disc[v], its back edge reaches a proper ancestor and the suffix remains with the outer block. If low[w]≥disc[v], no such edge exists and deletion of v separates this subtree from the outer part. Popping through vw takes exactly its block's pending edges and excludes already completed blocks. This boundary argument applies at arbitrary depth; the triangle is one trace. Bridges are one-edge blocks; singleton conventions must be explicit. Store edges, not merely vertices, to split blocks at cuts correctly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete DFS block decomposition

Use edge IDs, allowing parallel edges to distinguish the parent edge from another edge to the same parent. The chapter's base input is finite undirected simple graphs without loops.

Start DFS at every unvisited vertex. Emit an isolated vertex separately as a singleton block. Popping cannot depend on cut[v] being true: a one-child root must still emit its final block.

On each return, low values determine articulation points, bridges, and block boundaries; the following procedure implements those rules edge by edge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS: scan edges and recurse

The disc, low, cut arrays and edge stack are shared across calls; `finish_child` handles one completed child subtree.

```
DFS(v, parent_edge):
    disc[v]=low[v]=++clock; children=0
    for (w,e) in adjacency[v]:
        if e == parent_edge: continue
        if disc[w] is undefined:
            children++; stack.push(e); DFS(w,e)
            finish_child(v,w,e,parent_edge)
        else if disc[w] < disc[v]:
            stack.push(e); low[v]=min(low[v],disc[w])
    if parent_edge is absent: cut[v]=(children>=2)
```

Only the descendant pushes a back edge. Edge IDs preserve a parallel edge to the parent.

Child return: bridges and block boundaries

`finish_child(v,w,e,parent_edge)` receives the just-completed tree edge e=vw; the stack is shared throughout DFS.

```
finish_child(v,w,e,parent_edge):
    low[v]=min(low[v],low[w])
    if low[w] >= disc[v]:
        if parent_edge exists: cut[v]=true
        B=[]
        repeat: z=stack.pop(); B.append(z)
        until z==e
        emit B
    if low[w] > disc[v]: emit bridge e
```

The pop includes e. The root's child count determines whether it is a cut vertex, independently of block output.

Why the emitted blocks are maximal

The stack contains discovered edges not yet assigned to completed blocks. A subtree's pending edges form a suffix; already completed internal blocks have been removed. At low[w]≥disc[v], subtree w reaches the earlier graph only through v. Any connected larger subgraph crossing that boundary would separate on deleting v, so cannot be a larger cut-free block. At low[w]<disc[v], an ancestor bypass exists and the upper boundary has not yet been reached, so retain the suffix.

An emitted suffix has no internal cut z. If z≠v separated a child subtree from the rest, the corresponding low≥disc[z] event would have popped those edges earlier. They could not remain in this suffix. For z=v the emitted set contains one child subtree plus v, and deleting v leaves that subtree's DFS tree connected. Hence the emitted set is cut-free and cannot merge outward, proving maximality. Bridge singletons follow the stated convention.

Each edge is pushed and popped once and each adjacency entry scanned once. Time is O(n+m) and stack/array space O(n+m), including all emitted block edges rather than only the cut set.

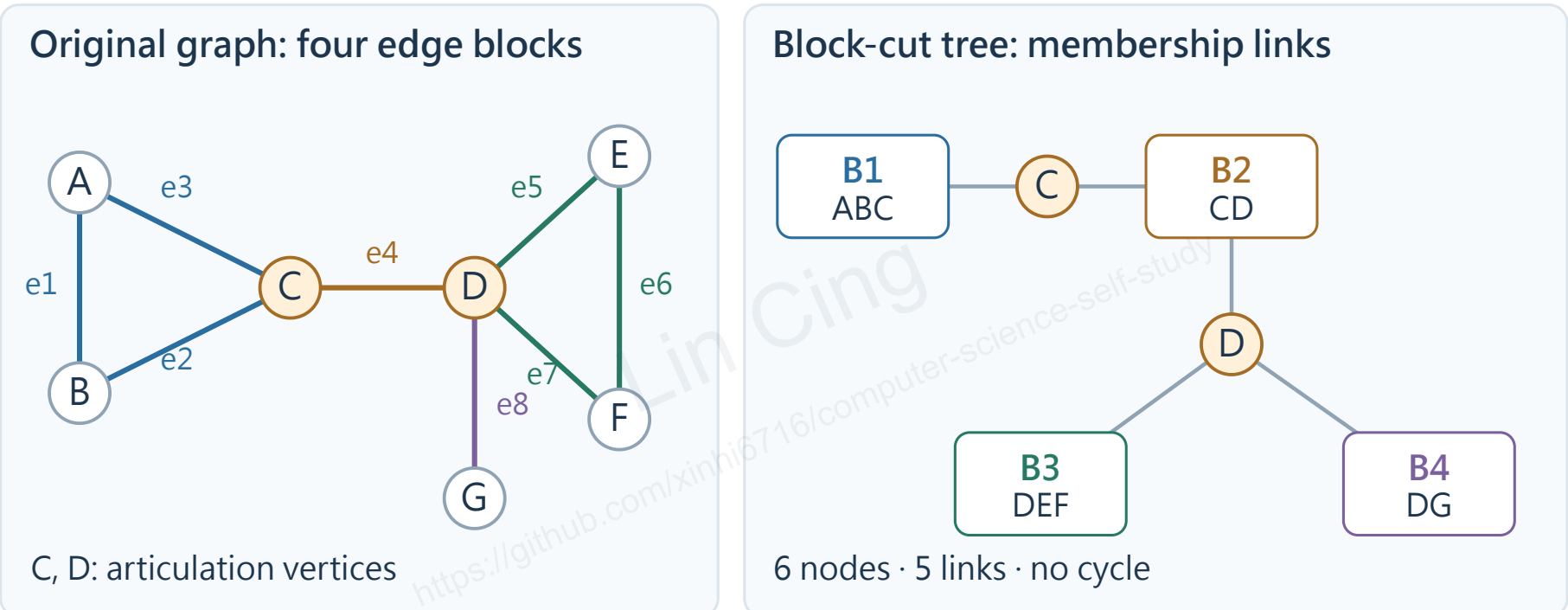
Block-cut trees and shared vertices

Create one node per block and one per articulation, with incidence edges for membership. For a connected graph this bipartite graph is connected. A cycle would provide bypasses around every supposed cut on it, permitting adjacent blocks to merge into a larger cut-free subgraph, contradicting maximality. Thus it is a tree; disconnected graphs give a forest.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From the original graph to its block-cut tree

Rectangles represent whole blocks; orange circles represent articulation vertices.

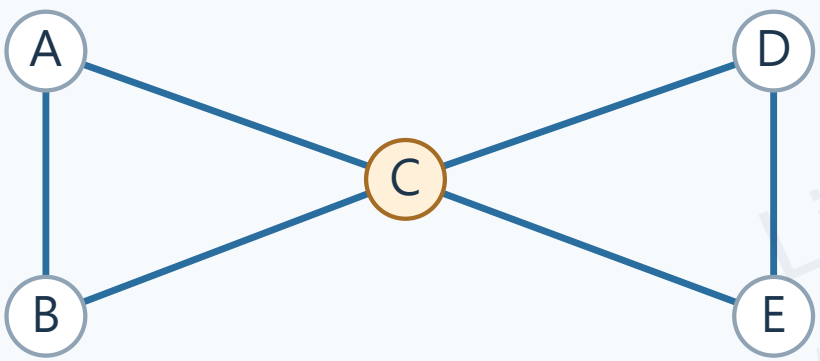


Membership links: B_1–C–B_2–D, D–B_3, D–B_4. C belongs to two blocks, D to three. Each original edge belongs to exactly one block.

No bridges does not imply no articulation

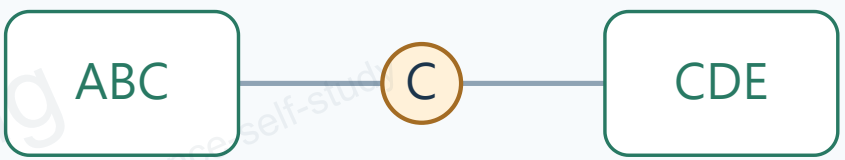
The triangles share C; compare the effects of deleting an edge and deleting a vertex.

Every edge lies on a cycle



No bridges; C separates AB from DE

Two vertex blocks; one cut



Delete all bridges: still one edge component.

Split at articulation C: two vertex blocks.

The two decompositions answer different questions.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Compare edge and vertex decompositions

Deleting any one edge leaves its endpoints connected around their triangle. Deleting C instead leaves the two disjoint edges AB and DE. Therefore C is an articulation even though no edge is a bridge. Bridge deletion leaves one edge-biconnected component; vertex-block decomposition gives ABC and CDE, sharing C.

A bridge tests whether one edge can be bypassed. An articulation point tests whether all paths between two remaining regions must use one vertex; these are different tests.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundaries and checks

Finite connected simple graphs with n−1 edges are already acyclic. Prefix-free words need not have equal lengths. Huffman optimizes within fixed-frequency prefix codes; other compression families require separate comparison. DFS roots use child counts. Block-cut trees contain block and cut nodes rather than treating every original edge as a block. Exercises and solutions give further derivations. These invariants and cost models support later spanning-tree, flow, and indexing methods.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 12.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study